61. Making Node.js Multithreaded: End-to-End with Diagrams

A deeper, step-by-step follow-up to **#20 (single vs multi thread)**, **#21 (worker threads)** and **#56 (async + multithreading)**.

> **In one line:** Node runs your JavaScript on **one main thread**. For **heavy CPU work** (maths, image resize, PDF, hashing, big loops), move the work to **worker threads**. They run on **other CPU cores at the same time**, so the main thread stays free to answer requests.

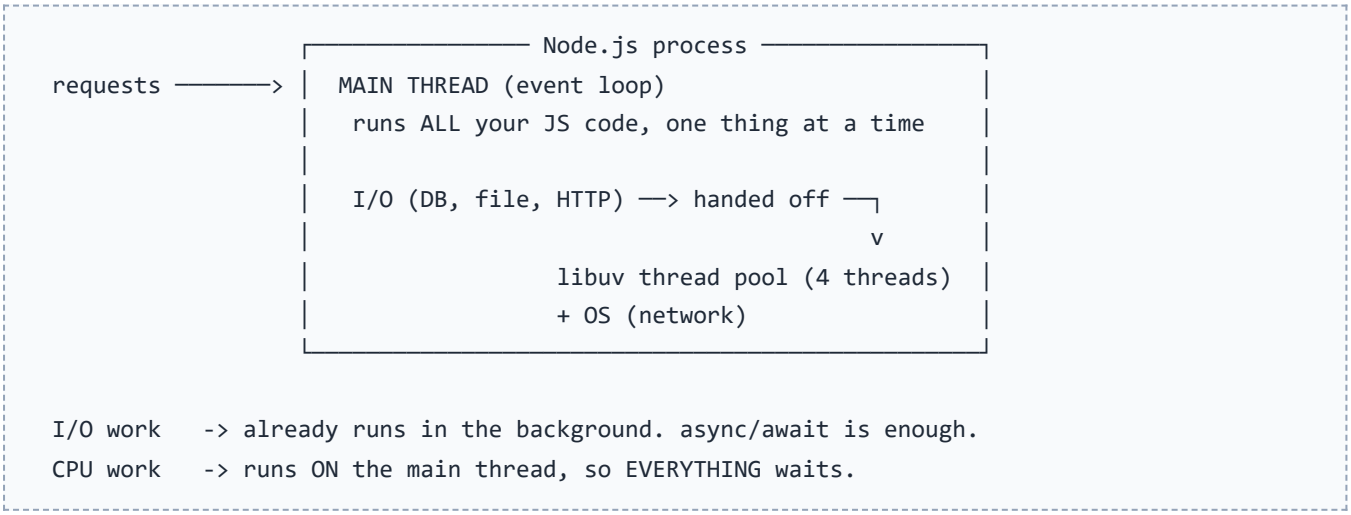
PART 1: The problem

1. Restaurant example

A restaurant has **one waiter** (the main thread).
Ordering food (I/O: DB, API, file) is fine. The waiter gives the order to the kitchen and serves other tables while it cooks. That is **async**.
But if the waiter has to **personally cut 100 kg of onions** (CPU work), nobody else gets served until they're done.

Fix: hire **helpers** (worker threads) to cut onions in the back. The waiter keeps serving.

2. What happens inside Node



3. Proof: one heavy request blocks everyone

```
// server.js (plain Node)
const http = require('http');

function fib(n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }  // pure CPU work

http.createServer((req, res) => {
  if (req.url === '/heavy') return res.end(`fib = ${fib(42)}`);   // about 3 seconds of CPU
  res.end('hello');                                               // instant
}).listen(3000);
```

```
time ──────────────────────────────────────────────────────────>
                0s                3s
User A /heavy   [████████ fib(42) ████████] done
User B /hello   .............waiting.......[x] done at 3s   <-- should be instant!
User C /hello   .............waiting.......[x] done at 3s

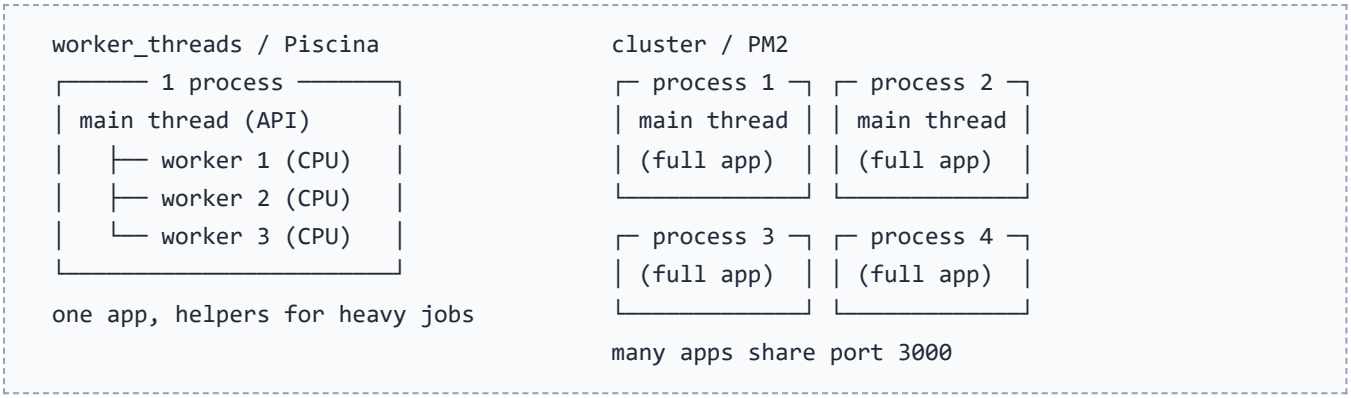
ONE thread, so B and C wait behind A's calculation.
```

Important: `async` / `await` does **not** fix this. Wrapping `fib(42)` in an async function still runs it on the main thread. Async helps only for **waiting** (I/O), not for **calculating** (CPU).

PART 2: The 4 ways to use more cores

4. Options at a glance

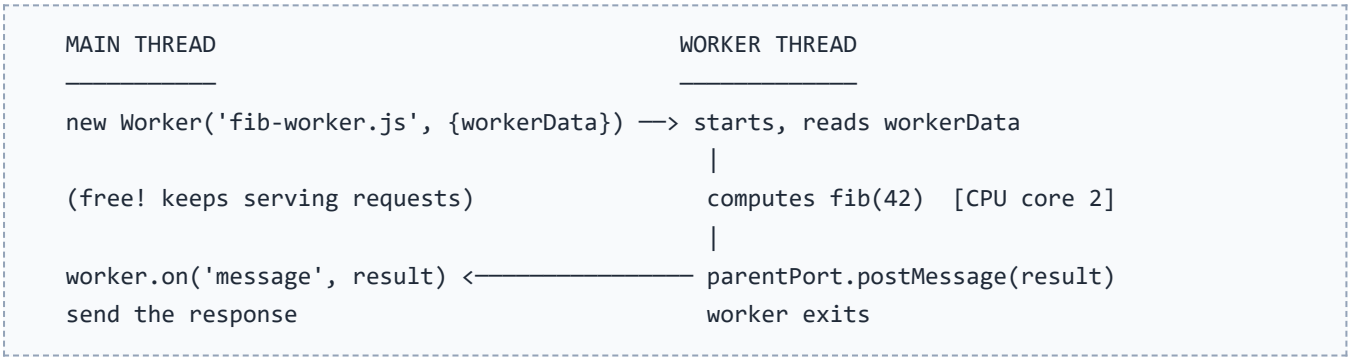
Way	What it is	Use for
worker_threads (built in)	Extra JS threads inside the **same process**	Heavy calculations in one request
Piscina (npm)	A **pool** of worker_threads, reused	The same, in a real server (best choice)
cluster / **PM2**	Many **copies of the whole app**, one per core	Handling more requests in total
libuv thread pool	Node's own hidden threads	Automatic for fs, crypto, zlib (raise `UV_THREADPOOL_SIZE`)



PART 3: worker_threads step by step

5. The basic idea: talk with messages

A worker is like a **separate room**. It **doesn't share variables** with the main thread. They talk by **sending messages**.



6. Step 1: the worker file

```
// fib-worker.js  (runs on another thread)
const { parentPort, workerData } = require('worker_threads');

function fib(n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }

const result = fib(workerData.n);    // heavy work happens HERE, not on the main thread
parentPort.postMessage(result);      // send the answer back
```

7. Step 2: the main file starts the worker

```
// server.js
const http = require('http');
const { Worker } = require('worker_threads');

function runFib(n) {
```

```
  return new Promise((resolve, reject) => {
    const worker = new Worker('./fib-worker.js', { workerData: { n } });
    worker.on('message', resolve);          // got the result
    worker.on('error', reject);             // crash inside the worker
    worker.on('exit', code => {
      if (code !== 0) reject(new Error(`worker stopped with code ${code}`));
    });
  });
}

http.createServer(async (req, res) => {
  if (req.url === '/heavy') {
    const result = await runFib(42);        // main thread is FREE while it waits
    return res.end(`fib = ${result}`);
  }
  res.end('hello');
}).listen(3000, () => console.log('http://localhost:3000'));
```

After: nobody waits

```
time ──────────────────────────────────────────────────────────>
                0s                3s
main thread     [A in][B][C] .... free .... [A out]
worker (core 2)     [████████ fib(42) ████████]
User B /hello      [x] instant
User C /hello         [x] instant
User A /heavy   ........................... [x] done at 3s
```

8. Step 3: split one big job across all cores

Example: **count prime numbers from 1 to 20 million**. One thread takes about 8 seconds. Split it into 4 parts, one per core.

```
   1 ──────────────────────────── 20,000,000
   ├── worker 1:   1 - 5M        ─┐
   ├── worker 2:   5M - 10M       ├─ all at the same time (4 cores)
   ├── worker 3:  10M - 15M       │
   └── worker 4:  15M - 20M      ─┘
                    |
          Promise.all  ->  add the 4 counts  ->  answer   (about 2 s instead of 8 s)
```

```
// prime-worker.js
const { parentPort, workerData } = require('worker_threads');

function isPrime(n) {
  if (n < 2) return false;
  for (let i = 2; i * i <= n; i++) if (n % i === 0) return false;
  return true;
}
```

```
let count = 0;
for (let n = workerData.from; n < workerData.to; n++) if (isPrime(n)) count++;
parentPort.postMessage(count);
```

```
// count-primes.js
const os = require('os');
const { Worker } = require('worker_threads');

function runWorker(from, to) {
  return new Promise((resolve, reject) => {
    const w = new Worker('./prime-worker.js', { workerData: { from, to } });
    w.on('message', resolve);
    w.on('error', reject);
  });
}

async function main() {
  const MAX = 20_000_000;
  const cores = os.cpus().length;                 // e.g. 8
  const size = Math.ceil(MAX / cores);

  console.time('parallel');
  const parts = [];
  for (let i = 0; i < cores; i++) {
    parts.push(runWorker(i * size, Math.min((i + 1) * size, MAX)));
  }
  const counts = await Promise.all(parts);         // wait for ALL workers
  console.log('primes:', counts.reduce((a, b) => a + b, 0));
  console.timeEnd('parallel');
}
main();
```

PART 4: Worker pool (Piscina), the real-world way

9. Why a pool?

> Hiring a new helper for every onion (a `new Worker()` per request) is slow: each start costs about **30-50 ms** and some memory. If 1,000 requests come in, you get 1,000 threads and the server dies.
> A **pool** = a **fixed team of helpers** (e.g. 4) who stay. Extra jobs **wait in a queue**.

```
  requests ──>  ┌──── Piscina pool ──────────────────────┐
  job 1 ──────>│  worker 1 [busy: job 1]                │
  job 2 ──────>│  worker 2 [busy: job 2]                │
  job 3 ──────>│  worker 3 [busy: job 3]                │
  job 4 ──────>│  worker 4 [busy: job 4]                │
  job 5 ──────>│  queue: [job 5] [job 6] ...  (waits)   │
  job 6 ──────>└────────────────────────────────────────┘
                  workers are REUSED, no start-up cost each time
```

```
npm install piscina
```

```
// fib.worker.js: just export a function
function fib(n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
module.exports = ({ n }) => fib(n);
```

```
// server.js
const path = require('path');
const Piscina = require('piscina');

const pool = new Piscina({
  filename: path.resolve(__dirname, 'fib.worker.js'),
  maxThreads: 4,                      // default is based on the CPU cores
});

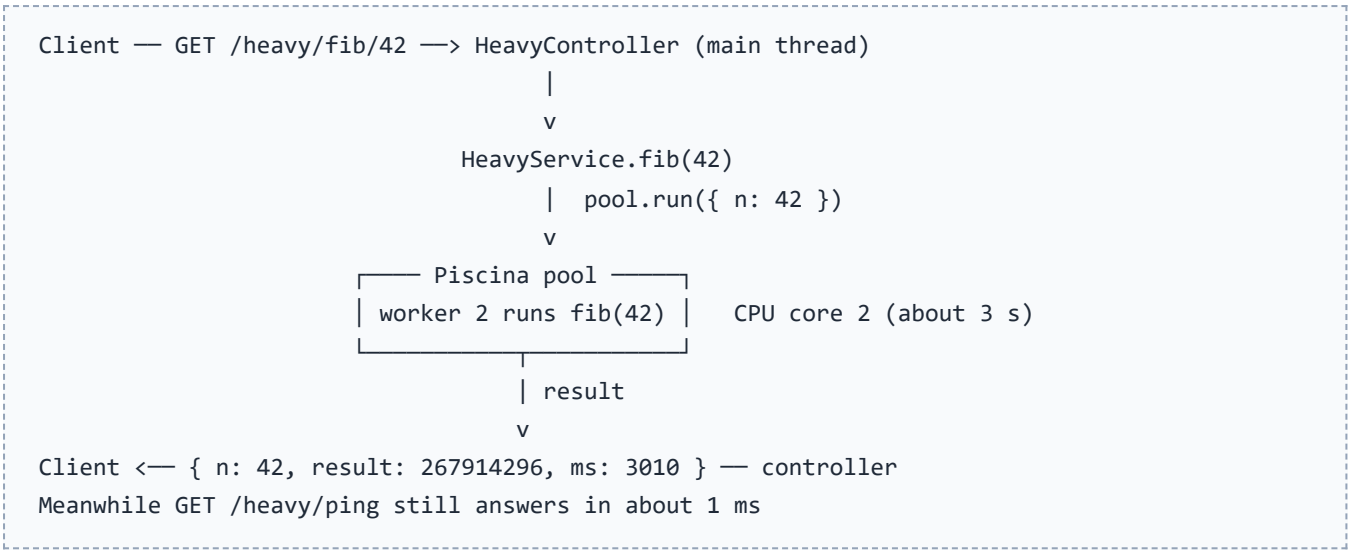
const result = await pool.run({ n: 42 });   // runs on a free worker, returns a Promise
```

PART 5: End to end in NestJS

10. Project structure

```
src/
  ├── heavy/
  │     ├── heavy.module.ts
  │     ├── heavy.controller.ts      GET /heavy/fib/:n  and  GET /heavy/ping
  │     ├── heavy.service.ts         owns the Piscina pool
  │     └── workers/
  │           └── fib.worker.ts      the CPU work (runs on worker threads)
  └── app.module.ts                  imports HeavyModule
```

The full request flow



11. The worker file

```
// src/heavy/workers/fib.worker.ts
function fib(n: number): number {
  return n < 2 ? n : fib(n - 1) + fib(n - 2);
}

// Piscina calls the default export with the data from pool.run()
export default function ({ n }: { n: number }): number {
  return fib(n);
}
```

12. The service (owns the pool)

```
// src/heavy/heavy.service.ts
import { Injectable, OnModuleDestroy } from '@nestjs/common';
import { join } from 'path';
import Piscina from 'piscina';

@Injectable()
export class HeavyService implements OnModuleDestroy {
  // __dirname = dist/heavy at runtime, so this points to the COMPILED .js file
  private readonly pool = new Piscina({
    filename: join(__dirname, 'workers', 'fib.worker.js'),
    maxThreads: 4,
  });

  async fib(n: number) {
    const start = Date.now();
    const result: number = await this.pool.run({ n });
    return { n, result, ms: Date.now() - start };
  }
```

```
  async onModuleDestroy() {
    await this.pool.destroy();          // close the threads on shutdown (#27 lifecycle)
  }
}
```

13. Controller and module

```
// src/heavy/heavy.controller.ts
import { Controller, Get, Param, ParseIntPipe } from '@nestjs/common';
import { HeavyService } from './heavy.service';

@Controller('heavy')
export class HeavyController {
  constructor(private readonly heavy: HeavyService) {}

  @Get('fib/:n')
  fib(@Param('n', ParseIntPipe) n: number) {
    return this.heavy.fib(n);           // CPU work goes to the pool
  }

  @Get('ping')
  ping() {
    return { ok: true, time: new Date().toISOString() };   // proves the main thread is free
  }
}

// src/heavy/heavy.module.ts
@Module({ controllers: [HeavyController], providers: [HeavyService] })
export class HeavyModule {}
```

Common error: "Cannot find module .../fib.worker.js". Workers run the **compiled JS** in `dist/`, not the `.ts` file. Run `npm run build` / `start:dev` so `dist/heavy/workers/fib.worker.js` exists, and always use `join(__dirname, ...)` with a `.js` extension.

14. Test it

```
# terminal 1
npm run start:dev

# terminal 2: start a heavy request
curl http://localhost:3000/heavy/fib/42

# terminal 3: at the same time, this still answers instantly
curl http://localhost:3000/heavy/ping
```

```
# load test: 8 heavy requests at once
npx autocannon -c 8 -a 8 http://localhost:3000/heavy/fib/40
```

Test	Without workers	With Piscina (4 threads)
`/ping` during a heavy request	Waits about 3 s	**About 1 ms**
4 heavy requests at once	About 12 s (one after another)	**About 3 s** (4 cores in parallel)
8 heavy requests at once	About 24 s	About 6 s (4 run, 4 queue)

> Numbers are approximate and depend on your CPU. The pattern is what matters: **the main thread never freezes**, and heavy jobs run **in parallel** up to the number of cores.

PART 6: More tools and rules

15. cluster / PM2: more copies of the whole app

```
# easiest way: PM2 starts one copy of the app per CPU core
npm install -g pm2
pm2 start dist/main.js -i max      # "-i max" = one process per core
pm2 list
```

```
                        ┌──> app copy 1 (core 1)
  requests -> port 3000 ├──> app copy 2 (core 2)     each copy has its own
                        ├──> app copy 3 (core 3)     memory: no shared variables,
                        └──> app copy 4 (core 4)     share state through Redis / DB
```

In **Kubernetes** (#53), you usually skip cluster/PM2 and run **more pods** instead (1 process per pod).

16. Sharing memory (advanced)

Normally messages are **copied**. For big data (e.g. a 100 MB image), copying is slow. Options:

- **Transfer** an `ArrayBuffer`: ownership moves to the worker with no copy. `postMessage(buf, [buf])`
- **SharedArrayBuffer + Atomics**: both threads see the same memory. It's powerful but needs care to avoid race conditions.

17. When to use what

Your work is...	Example	Use
Waiting (I/O)	DB query, API call, reading a file	**async/await** only. Workers don't help.
Heavy CPU, inside a request	Image resize, PDF, big JSON, hashing, maths	**Piscina** (worker pool)
Heavy CPU, can run later	1,000 emails, video encoding, reports	**BullMQ** + worker pods (#57)
Too many requests for one core	High traffic API	**PM2 cluster** or more **K8s pods**
Huge Python / ML work	Model training, 10,000 images	**KubeRay** (#55, #59)

18. Common mistakes

Mistake	Result	Fix
Thinking `async` makes CPU work parallel	Server still freezes	Use worker threads
`new Worker()` on every request	Slow, memory explodes under load	Use a pool (Piscina)
More threads than CPU cores	Slower (threads fight for cores)	`maxThreads` <= `os.cpus().length`
Using workers for DB / HTTP calls	Extra cost, no gain	Keep I/O on the main thread with async
Expecting workers to share variables	Values are `undefined` / stale	Send data by message, or use SharedArrayBuffer
Worker path pointing to `.ts`	"Cannot find module"	Point to the compiled `.js` in `dist`
No error / exit handler	Request hangs forever	Handle `error` and `exit` (Piscina does this for you)

19. Summary

- Node runs JS on **one main thread**. **I/O is already async**, but **CPU work blocks everyone**.
- `async` / `await` does **not** make CPU work parallel.
- **worker_threads**: create a `Worker`, pass `workerData`, get the result with `postMessage`.
- **Split** big jobs into parts (one per core) and combine them with `Promise.all`.
- **Piscina** = a reusable worker pool, the right way in a server. In NestJS: the pool lives in a service, `pool.run()` in a method, `destroy()` on shutdown.
- **PM2 cluster** / more **K8s pods** = more copies of the whole app for more traffic.
- Rule: **waiting, use async. Calculating, use worker threads. Later or huge, use a queue (BullMQ) or KubeRay.**